



COLLEGE OF COMPUTING AND INFORMATICS

PUTRAJAYA CAMPUS

ASSIGNMENT PART 2

SEMESTER 2 2025/2026

SUBJECT CODE: CSEB5223

SUBJECT: SOFTWARE CONSTRUCTION AND METHODS

NAME OF MEMBERS:

No.	Full Name	Student ID
1.	Wan Jehan Farouq Bin Wan Ahmed	BSW01084722
2.	Thaqif Bin Nabil	BSW01084719
3.	Muhammad Haikal Sharafuddin Bin Shahrol Azam	BSW01084695
4.	Muhammad Danesh Hakimi Lou	BSW01084693

Table of Contents

1.0 Introduction.....	1
2.0 Use Case Design	1
2.1 Actors	1
2.2 Use Case Description.....	1
2.3 Use Case Diagram.....	2
3.0 Class Diagram.....	3
4.0 Documentation Formatting Standard	4
4.1 General Document Format.....	4
4.2 Section Formatting Rules.....	4
4.3 Diagrams Formatting	5
4.4 Code documentation Format.....	5
4.5 Version Control Documentation.....	6
4.6 Coding File Formatting Standard.....	6
4.7 General Code Formatting Rules.....	6
4.8 Commenting Rules.....	7
4.9 Code Cleanliness Rules.....	7
4.10 Define Naming Convention	8
4.11 Naming Dictionary.....	9
5.0 Define Function and Methods Modularity.....	10
5.1 Introduction.....	10
5.2 Function-Level Modularity	10
5.3 Module-Level Modularity.....	11
6.0 Define Continuous Integration Rules.....	13
7.0 Course Profile	14
7.1 Introduction.....	14
7.2 Screenshots	14
7.2.1 Add Course.....	14
7.2.2 Search Course	15
7.2.3 Edit Course.....	15
7.2.4 Delete Course.....	16
7.2.5 View all Courses	16
7.3 Analysis of Software Construction Best Practices and Challenges	17
7.3.1 Best Practices	17
7.4 Challenges.....	19

8.0 Student Profile	20
8.1 Introduction.....	20
8.2 Screenshots	20
8.2.1 Add Student.....	20
8.2.2 View All Students.....	21
8.2.3 Search Student	21
8.2.4 Edit Student.....	22
8.2.5 Delete Student.....	23
8.3 Analysis of Software Construction Best Practices and Challenges	24
8.3.1 Best Practices	24
8.3.2 Challenges.....	25
8.4 System Integration	25
9.0 Student Relationship and Middleware API.....	26
9.1 Introduction.....	26
9.2 Screenshots	26
9.2.1 Add Course to Student (addCourse()).....	26
9.2.2 Add Student to Course (addStudent())	27
9.2.3 Find Student's Course (findCourse())	27
9.2.4 List All Courses of a Student (listCourses())	28
9.2.5 Find Student in a Course.....	28
9.2.6 List All Students in a Course.....	29
10.0 Error Handling and Edge Cases	30
10.1 Overview.....	30
10.1.1 Student or Course Not Found.....	30
10.1.2 Student Without Assigned Course.....	30
10.1.3 Course Without Assigned Students	31
10.1.4 Duplicate Assignment Prevention	32
10.1.5 Invalid Operations Handling.....	33
11.0 Middleware API Implementation	34
11.1 Overview.....	34
11.1.1 Data Caching	34
12.0 System Integration	35
12.1 Overview.....	35
12.1.1 Main Menu Interface.....	35
12.1.2 Integration Flow	36
12.1.3 System Features	36

1.0 Introduction

The Student Learning Management System (SLMS) is designed to support academic activities between students and lecturers. The system enables lecturers to manage course materials and assignments, while allowing students to access learning resources, submit assignments, and view grades

The overall system design is represented using Use Case and Class diagrams to illustrate system functionality and object-oriented structure

2.0 Use Case Design

2.1 Actors

The SLMS consists of two main actors:

- **Student** – A registered user who enrolls in courses, views materials, submits assignments, and views grades.
- **Lecturer** – A user responsible for managing course materials, creating assignments, and grading student submissions.

2.2 Use Case Description

The Use Case Diagram illustrates how users interact with the SLMS system.

Student Functionalities:

- Register and login
- View course materials
- View assignments
- Submit assignments
- View grades

Lecturer Functionalities:

- Login
- Upload course materials
- Create and update assignments
- View student submissions
- Grade and publish results

2.3 Use Case Diagram

The Use Case Diagram illustrates the interaction between two primary actors, Student and Instructor, with the SLMS system. Students can register, log in, view materials, submit assignments, and view grades. Instructors can upload materials, create assignments, view submissions, and grade student work. Certain use cases such as “Submit Assignment” include “Upload File” to represent dependent functionality. The diagram defines system boundaries and actor interactions clearly.

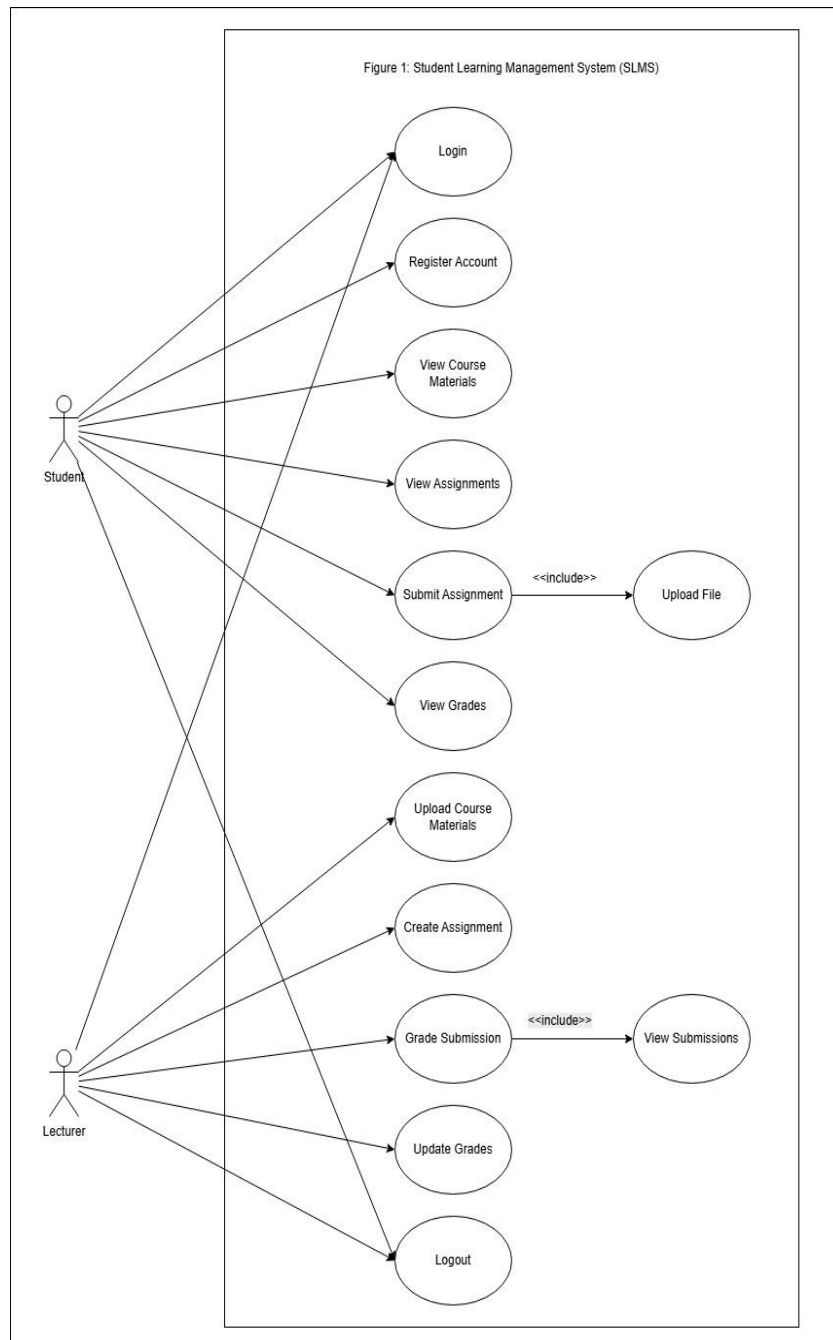


Figure 1: Use Case Diagram

3.0 Class Diagram

The Class Diagram represents the object-oriented structure of the SLMS. The system follows inheritance where Student and Instructor extend the User class. A Course aggregates Assignments and Materials. Each Assignment can have multiple Submissions. This structure promotes modularity, reusability, and scalability by clearly separating responsibilities across classes.

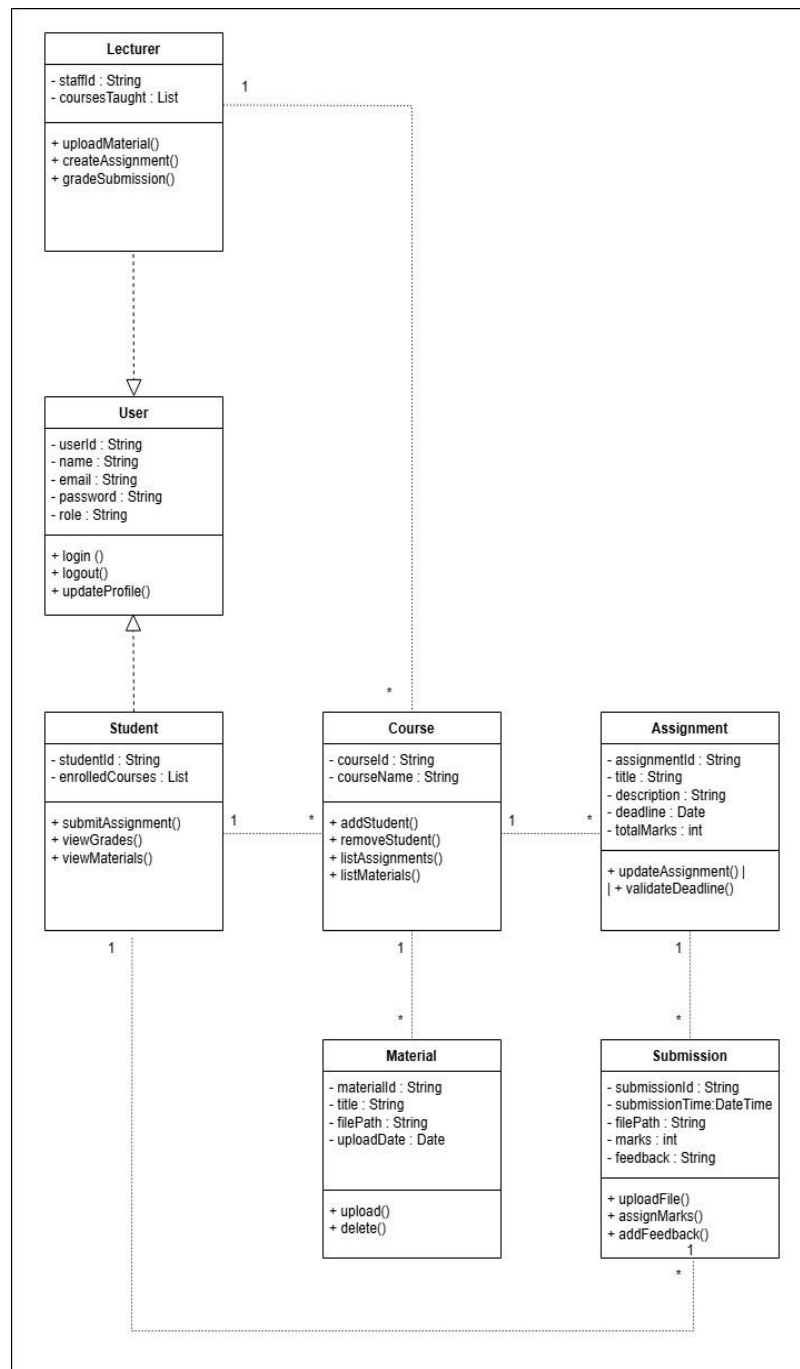


Figure 2: Class Diagram

4.0 Documentation Formatting Standard

To ensure clarity, consistency, and professionalism in the UniLearn project, the following documentation formatting standards will be used.

4.1 General Document Format

Item	Standard
Font	Times New Roman
Font Size	12 pt (body text)
Headings	14 - 16 pt, Bold
Line Spacing	1.5
Margins	1 inch (2.54 cm)
Alignment	Justified
Page Number	Bottom Center
File Format	PDF (.pdf)

4.2 Section Formatting Rules

Headings:

- Main Heading – Bold, Capitalize Each Word
- Subheading – Bold
- Sub-subheading – Bold + Numbered (e.g., 2.1.1)

Example:

2.0 Main Heading

2.1 General Format

2.1.1 Font Style

4.3 Diagrams Formatting

All diagrams (Use Case, Class Diagram, etc.) must:

- Use consistent font style
- Include diagram title below the figure
- Include figure numbering

Example:

Figure 1: UniLearn Use Case Diagram

4.4 Code documentation Format

All code must include:

- File Header Comment

Example:

```
/*  
  
*   Project: UniLearn  
  
*   Course: CSEB5223 - Software Construction & Methods  
  
*   Author: [Student Name]  
  
*   Date Created: [DD/MM/YYYY]  
  
*   Description: [Brief explanation of file purpose]  
  
*/
```


4.5 Version Control Documentation

Every update must include:

- Version Number
- Date
- Updated By
- Description of Changes Example:

Version	Date	Author	Description
1.0	19/2/2026	Thaqif	Initial Draft
1.1	20/2/2026	Haikal	Updated Formatting

4.6 Coding File Formatting Standard

To maintain clean, readable, and maintainable code, the following coding file formatting standards will be used for the UniLearn project.

4.7 General Code Formatting Rules

Rule	Standard
Indentation	4 spaces
Maximum Line Length	100 – 120 characters
Braces Style	Opening brace on same line
Spacing	One space after keywords
Blank Lines	Separate logical sections

4.8 Commenting Rules

- Use comments to explain “why”, not obvious code
- Avoid unnecessary comments
- Inline comments only when necessary

Example:

```
// Check if student exists before enrollment
```

```
if (student != null) {    enroll(student);  
}
```

4.9 Code Cleanliness Rules

- No unused variables
- No duplicate code
- Keep methods short (one responsibility per method)

4.10 Define Naming Convention

1. Class Names	• Use PascalCase • Must be nouns • Example: ○ Student ○ CourseRegistration ○ UserAccount
2. Variable Names	• Use camelCase • Must be meaningful • No short unclear names like x, abc • Example: ○ studentName ○ totalCredits ○ loginStatus
3. Method Names	• Use camelCase • Must start with a verb • Example: ○ calculateGPA() ○ registerCourse() ○ validateLogin()
4. Constants	• Use UPPER_CASE • Separate words with underscore • Example: ○ MAX_CREDIT_LIMIT ○ DEFAULT_PASSWORD

5. File Names	• Match class name • One class per file • Example: ○ Student.java ○ Course.java
6. Database Naming (if applicable)	• Tables: • Singular noun • Lowercase • Example: ○ student ○ course ○ Columns: • snake_case • Example: ○ student_id ○ course_code

4.11 Naming Dictionary

Term	Meaning
Student	A registered user enrolled in the institution
Lecturer	A user who teaches courses in the institution
Course	A subject offered in the system
Enrollment	The process of a student registering for a course
Admin	System administrator managing users

5.0 Define Function and Methods Modularity

5.1 Introduction

In the SLMS system, functions and methods are designed using modular programming principles. A function is a block of code that performs a specific task. A method is a function that belongs to a class in object-oriented programming. Modularity refers to organizing these functions and methods into independent modules where each module is responsible for one specific functionality.

5.2 Function-Level Modularity

Each function in the SLMS performs only one clear responsibility. This follows the **Single Responsibility Principle (SRP)**.

Examples of function-level modularity:

- `loginUser()` → validates user credentials only.
- `submitAssignment()` → handles assignment submission only.
- `calculateFinalGrade()` → computes final grades only.
- `uploadMaterial()` → manages file upload for learning materials only.

No function performs multiple unrelated tasks. This avoids overly complex methods and improves readability.

5.3 Module-Level Modularity

The SLMS system is divided into logical modules. Each module groups related functions together and separates them from other functionalities.

1. Authentication Module

Responsible for **user login, logout, and verification.**

Functions:

- registerUser()
- loginUser()
- logoutUser()
- verifyTwoFactorCode()

2. User Management Module

Responsible for **handling user account information.**

Functions:

- createUser()
- updateUser()
- deleteUser()
- getUserById()

3. Assignment Module

Responsible for **creating and managing assignments.**

Functions:

- createAssignment()
- updateAssignment()
- deleteAssignment()
- validateDeadline()

4. Submission Module

Responsible for **handling student assignment submissions.**

Functions:

- submitAssignment()
- checkLateSubmission()
- getSubmissionsByAssignment()

5. Grading Module

Responsible for **evaluating and grading submissions.**

Functions:

- assignMarks()
- addFeedback()
- calculateFinalGrade()
- publishGrades()

6. Material Module

Responsible for **managing learning materials.**

Functions:

- uploadMaterial()
- deleteMaterial()
- listMaterials()

6.0 Define Continuous Integration Rules

1. Version Control	• Use Git • Repository hosted on GitHub • All team members must commit changes regularly
2. Branching Strategy	• No direct push to main • Pull request required • Code review required before merging
3. Commit Rules	• Clear commit messages • Format: [MODULE] Short description
4. Automated Testing	Every push triggers: • Code compilation • Unit tests • If tests fail → merge not allowed
5. Build Verification	Before merging: • No syntax errors • No failing tests • No unresolved conflicts
6. Code Quality Rules	• Follow naming conventions • No duplicate code • Proper comments • No hardcoded sensitive data

7.0 Course Profile

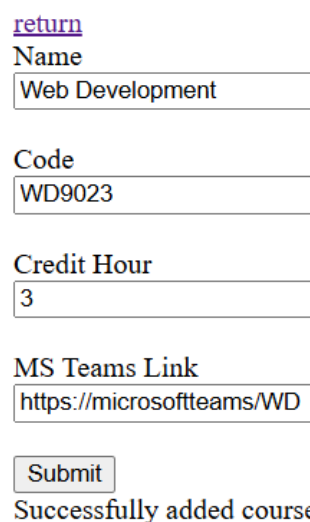
7.1 Introduction

The UniLearn system is developed primarily using HTML to structure and present the user interface of the application. To comply with the requirement of applying object-oriented programming (OOP) practices, JavaScript is integrated into the system to handle logic and functionality. Through JavaScript, key features are implemented using OOP concepts such as classes, objects, and modular design, ensuring the system is well-structured, reusable, and maintainable while still fulfilling the functional requirements.

7.2 Screenshots

7.2.1 Add Course

ADD NEW COURSE



[return](#)

Name

Code

Credit Hour

MS Teams Link

Successfully added course.

Figure 3: Add new course

The screenshot shows the “Add New Course” section of the UniLearn system, where users can create a new course. The interface includes input fields such as course name, course code, credit hours and Microsoft Teams Link along with a submission button to save the course details. A simple successful message will be displayed upon submission.

7.2.2 Search Course

SEARCH COURSE

[return](#)
WD9023

Web Development

Code: WD9023
Credit Hour(s): 3
MS Teams Link: <https://microsoftteams/WD>

Figure 4: Section to search for courses

The screenshot shows the “Search Course” section of the UniLearn system, where users can find existing courses quickly. The interface includes a search bar that allows users to enter keyword such as course code, and press “Enter” to display relevant results.

7.2.3 Edit Course

EDIT COURSE

[return](#)
AI1234

AI

Code: AI1234
Credit Hour(s): 3
MS Teams Link: <https://microsoftteams/ai>

Name
Artificial intelligence

Credit Hour
2

MS Teams Link
<https://microsoftteams/ai>

Successfully edited course

Figure 5: Section to edit course details

The screenshot shows the “Edit Course” section of the UniLearn system, where users can update existing course information. First, users need to enter the course code that they want to edit. Then, the interface will display editable fields such as course name, credit hours and MS Teams Link along with a submit button to save the changes. A simple successful message will be displayed upon submission.

7.2.4 Delete Course

DELETE COURSE

[return](#)
SP110

Sports

Code: SP110

Credit Hour(s): 2

MS Teams Link: N/A

Would you like to delete this course?

[return](#)
SP110

Successfully removed course.

Figure 6: Section to delete course

The screenshot shows the “Delete Course” section of the UniLearn system, where users can remove an existing course from the system. The system will allow users to search for a specific course by entering the course code, along with a confirmation prompt to confirm the action. A simple successful message will be displayed upon deletion.

7.2.5 View all Courses

VIEW ALL COURSES

[return](#)

[1] Web Development	[2] Artificial intelligence
Code: WD9023	Code: AI1234
Credit Hour(s): 3	Credit Hour(s): 2
MS Teams Link: https://microsoftteams/WD	MS Teams Link: https://microsoftteams/ai

Figure 7: Section to view all courses

The screenshot shows the “View All Courses” section of the UniLearn system, where users can see a complete list of all available courses. The interface will display course details such as course name, course code, credit hours and MS Teams Link in a structured format. This section allows users to easily browse and review all course information within the system.

7.3 Analysis of Software Construction Best Practices and Challenges

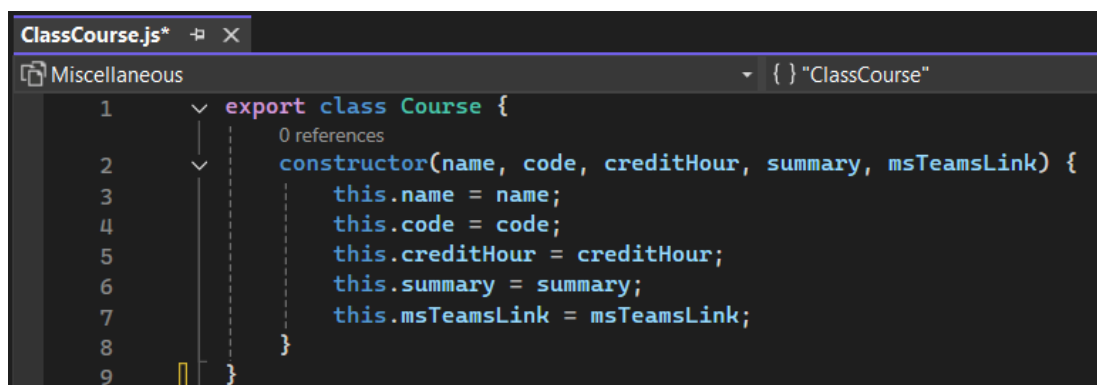
7.3.1 Best Practices

i) Implementation of Core Functionality

The UniLearn system successfully supports essential operations such as adding, viewing, searching, editing, and deleting course profiles. For example, users can input a new course in the “Add New Course” section and immediately see it listed in the “View All Courses” section, showing that the system processes and displays data correctly.

ii) Use of Object-Oriented Programming (OOP)

JavaScript is used to implement OOP concepts such as classes and objects. For instance, a Course class is used to represent course data (course name, course code, credit hours, summary, ms teams link), this improves code organization and reusability:

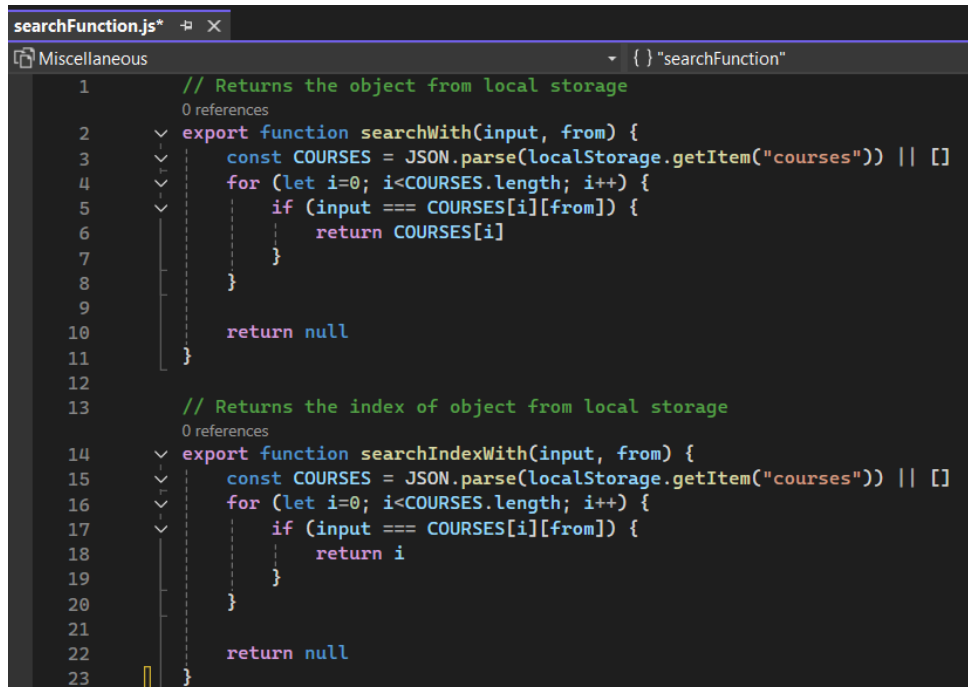


```
ClassCourse.js*  Miscellaneou  { } "ClassCourse"
1  export class Course {
2      0 references
3      constructor(name, code, creditHour, summary, msTeamsLink) {
4          this.name = name;
5          this.code = code;
6          this.creditHour = creditHour;
7          this.summary = summary;
8          this.msTeamsLink = msTeamsLink;
9      }
}
```

Figure 8: Course class (javascript)

iii) Consistent Documentation Standards

Each file includes header comments describing its purpose and functions. This helps developers understand how each part of the system works without confusion:



```
searchFunction.js*
Miscellaneous {} "searchFunction"

1 // Returns the object from local storage
  0 references
2 export function searchWith(input, from) {
3   const COURSES = JSON.parse(localStorage.getItem("courses")) || []
4   for (let i=0; i<COURSES.length; i++) {
5     if (input === COURSES[i][from]) {
6       return COURSES[i]
7     }
8   }
9
10  return null
11 }
12
13 // Returns the index of object from local storage
  0 references
14 export function searchIndexWith(input, from) {
15   const COURSES = JSON.parse(localStorage.getItem("courses")) || []
16   for (let i=0; i<COURSES.length; i++) {
17     if (input === COURSES[i][from]) {
18       return i
19     }
20   }
21
22  return null
23 }
```

Figure 9: Code commenting

iv) Clean and Organized Code Structure

The code follows consistent naming conventions such as camelCase for variables (e.g., creditHour) and PascalCase for classes (e.g., Course). Files are also organized logically, separating HTML structure and JavaScript logic, making the system easier to maintain.

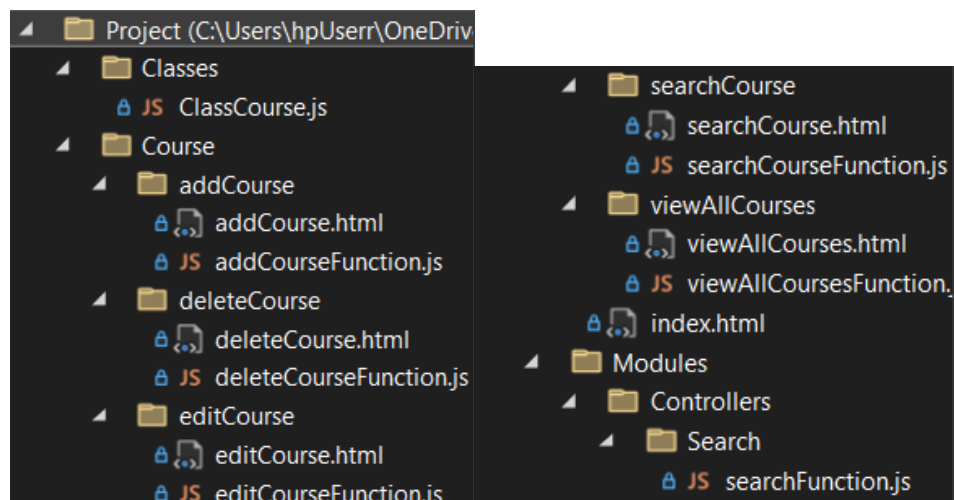


Figure 10: File organization

v) User-Friendly Interface Design

The interface is simple and intuitive. For example, in the “Search Course” section, users only need to enter the course code in a search bar and press “Enter” to retrieve results, making the system easy to use even for beginners.

vi) Basic Error Handling Implementation

The system includes input validation to prevent errors. For example, when searching a course, the system checks if required fields like course name and course code are entered properly before providing results, reducing the risk of invalid data.

7.4 Challenges

i) Integration Between HTML and JavaScript

Handling dynamic updates between HTML and JavaScript can be challenging. For example, after deleting a course, ensuring that the “View All Courses” list updates immediately without refreshing the page requires careful coding.

ii) Maintaining OOP Structure in JavaScript

Applying OOP concepts consistently can be difficult. For instance, ensuring that all course-related operations (add, edit, delete) are properly encapsulated within the Course class or related modules requires proper design planning.

iii) Ensuring Consistent Coding Standards

When multiple features are developed (e.g., Add Course, Search Course, Edit Course), maintaining consistent naming and formatting across all JavaScript files can be challenging, especially in group work.

8.0 Student Profile

8.1 Introduction

The Student Profile module is developed as part of the Student Learning Management System (SLMS). The objective of this module is to allow users to manually input student data and perform operations such as searching, editing, deleting, and displaying student information in a clear and organized format.

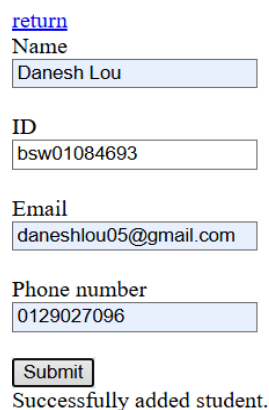
The system is implemented using HTML for interface design and JavaScript applying Object-Oriented Programming (OOP) concepts. This ensures that the system is modular, maintainable, and easy to use.

8.2 Screenshots

8.2.1 Add Student

The screenshot shows the “Add New Student” section of the system. Users can input student details such as name, student ID, email, and phone number. After clicking the submit button, the system displays a success message indicating that the student has been successfully added.

ADD NEW STUDENT



[return](#)

Name
Danesh Lou

ID
bsw01084693

Email
daneshlou05@gmail.com

Phone number
0129027096

Successfully added student.

Figure 11: Add new student section

8.2.2 View All Students

The screenshot shows the “View All Students” section where all stored student records are displayed. The system presents student information in a structured format including name, ID, email, and phone number.

VIEW ALL STUDENTS

[return](#)

[1] Danesh Lou

ID: bsw01084693

Email: daneshlou05@gmail.com

Phone Number: 0129027096

Figure 12: View all students' section

8.2.3 Search Student

The screenshot shows the “Search Student” function. Users enter a student ID and press enter to retrieve the student’s information. The system successfully displays the student details when a valid ID is provided.

SEARCH STUDENT

[return](#)

Enter Student ID:

Press ENTER

Adam Lew

ID: bsw01084693

Email: adam2003@gmail.com

Phone Number: 0129098890

Figure 13: Search Student's section

8.2.4 Edit Student

The screenshot shows the “Edit Student” function where users can modify student details except for the Student ID. After editing, the updated information is displayed to confirm the changes.

EDIT STUDENT

[return](#)

bsw01084693

Danesh Lou

ID: bsw01084693

Email: daneshlou05@gmail.com

Phone Number: 0129027096

Name

Adam Lew

ID (Cannot be edited)

bsw01084693

Email

adam2003@gmail.com

Phone Number

0129098890

Update Student

Successfully edited student

Figure 14: Edit Student's section

8.2.5 Delete Student

The screenshot shows the “Delete Student” function. The system displays the student details and asks for confirmation before deletion. After confirmation, the student is removed and a success message is shown.

DELETE STUDENT

[return](#)

bsw01084693

Danesh Lou

ID: bsw01084693

Email: daneshlou05@gmail.com

Phone Number: 0129027096

Would you like to delete this student?

[return](#)

bsw01084693

Successfully removed student.

Figure 15: Delete Student's section

8.3 Analysis of Software Construction Best Practices and Challenges

8.3.1 Best Practices

i) Implementation of Core Functionality

The system successfully implements all required operations including add, search, edit, delete, and view functions. Each function works correctly and produces the expected output.

ii) Use of Object-Oriented Programming (OOP)

The system uses a Student class to represent student data. This improves code organization, reusability, and maintainability.

iii) Modular Design

Each functionality is implemented using separate functions such as addStudent(), searchStudent(), editStudent(), and deleteStudent(). This follows the Single Responsibility Principle.

iv) Input Validation and Error Handling

The system includes validation to prevent invalid inputs such as empty fields and duplicate Student IDs. It also handles cases where a student is not found.

v) Data Storage and Management

Student data is stored in arrays and managed properly to prevent errors such as out-of-bound access.

vi) Clean Coding Standards

The code follows consistent naming conventions and proper formatting, making it easy to read and understand.

vii) User-Friendly Interface

The interface is simple and easy to navigate, allowing users to perform operations without confusion.

8.3.2 Challenges

i) Managing Data Consistency

Ensuring that student data remains consistent after editing and deletion required careful implementation.

ii) Preventing Duplicate Student IDs

Additional validation logic was needed to ensure that each student ID is unique.

iii) Dynamic Display Updates

Updating the displayed data after operations without refreshing the page required proper handling.

iv) Integration Between Modules

Integrating the Student Profile module with the Course Profile module required proper organization of files and functions.

v) Maintaining OOP Structure

Applying OOP concepts consistently across all functionalities required planning and design.

8.4 System Integration

The Student Profile module has been successfully integrated with the Course Profile module. Both modules are accessible within the same system through a navigation menu. This integration allows users to manage both course and student data within a single system, improving functionality and usability.

9.0 Student Relationship and Middleware API

9.1 Introduction

The Student Profile module is developed as part of the Student Learning Management System (SLMS). The objective of this module is to allow users to manually input student data and perform operations such as searching, editing, deleting, and displaying student information in a clear and organized format.

The system is implemented using HTML for interface design and JavaScript applying Object-Oriented Programming (OOP) concepts. This ensures that the system is modular, maintainable, and easy to use.

9.2 Screenshots

9.2.1 Add Course to Student (addCourse())

The screenshot shows the "Add Course to Student" function. The user first goes to Edit Course and enters a course code. The user then enters a student ID under "Add Student to Course" and presses the "Add" button to add the student to the course. A success message is shown upon completion.

EDIT COURSE

[return](#)
CSEB2222

Name
Web Development

Credit Hour
2

MS Teams Link
www.teams/webprog.com

Type
Core

Students enrolled

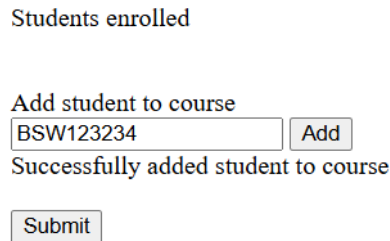
Add student to course
BSW019123234 Add

Submit

Figure 16: Add course to Student's section

9.2.2 Add Student to Course (addStudent())

The screenshot shows the "Add Student to Course" function. The system prompts for a course ID and a student ID, then adds the student to the course's assigned list. A success message is shown upon completion.



Students enrolled

Add student to course

BSW123234

Successfully added student to course

Figure 17: Add Student to course section

9.2.3 Find Student's Course (findCourse())

The screenshot shows the "Find Course by Student ID" function. The user first goes to View All Students and the list of all students is displayed. Below each student, the list of courses that student is enrolled in will appear. The user can also enter a student ID to find a specific student. A list of courses is shown.

VIEW ALL STUDENTS

[return](#)

[1] Rohani Binti Johan

ID: BSW123234

Email: rohani@uniten.com

Phone Number: 019123234

Courses taken

[2] Mobile App Dev | CSEB3333 | 3 | www.teams/mobiledev.com | core

Search course

CSEB3333

Figure 18: Find Student's course section

9.2.4 List All Courses of a Student (listCourses())

The screenshot shows the "List Courses for Student" function. The user goes to View All Students, where each student will show their registered courses below. The complete course list for each student is shown.

VIEW ALL STUDENTS

[return](#)

[1] Rohani Binti Johan

ID: BSW123234

Email: rohani@uniten.com

Phone Number: 019123234

Courses taken

[1] Web Development | CSEB2222 | 2 | www.teams/webprog.com | core

[2] Mobile App Dev | CSEB3333 | 3 | www.teams/mobiledev.com | core

Search course

Figure 19: List all courses of a student section

9.2.5 Find Student in a Course

The screenshot shows the "Find Student by Course ID" function. The user goes to View All Courses, where all courses will be listed. The user can enter a student ID in the text box below each course to find the specific student. A list of students is shown.

VIEW ALL COURSES

[return](#)

[1] Web Development

Code: CSEB2222

Credit Hour(s): 2

MS Teams Link: www.teams/webprog.com

Type: core

Students enrolled

[1] Rohani Binti Johan | BSW123234 | rohani@uniten.com | 019123234

Search student

Figure 20: Find student in a course section

9.2.6 List All Students in a Course

The screenshot shows the "List Students in Course" function. The user goes to View All Courses, where all courses will be displayed, and the students registered will be written under the specific course. The complete student list is shown.

VIEW ALL COURSES

[return](#)

[1] Web Development

Code: CSEB2222

Credit Hour(s): 2

MS Teams Link: www.teams/webprog.com

Type: core

Students enrolled

[1] Rohani Binti Johan | BSW123234 | rohani@uniten.com | 019123234

[2] Adam | BSW111111 | adam@gmail.com | 011223343

Search student

Figure 21: List all students in a course section

10.0 Error Handling and Edge Cases

10.1 Overview

Error handling is implemented to ensure the system remains stable and prevents invalid operations.

Implemented Error Checks:

10.1.1 Student or Course Not Found

Whenever a search operation is performed, the system verifies whether the student or course exists.

SEARCH COURSE

[return](#)

Course "cseb123" was not found.

Figure 22: Student or course not found

10.1.2 Student Without Assigned Course

When listing courses for a student:

If the student has no enrolled courses, the system displays:

VIEW ALL STUDENTS

[return](#)

[1] Danesh Lou

ID: BSW01084693

Email: daneshlou05@gmail.com

Phone Number: 0129027096

Courses taken

Search course

Figure 23: Student without assigned course

10.1.3 Course Without Assigned Students

When listing students in a course:

- If no students are assigned, the system displays:

VIEW ALL COURSES

[return](#)

[1] software construction

Code: csda1231312

Credit Hour(s): 1

MS Teams Link: <https://teams.microsoft.com/l/meetup-join/>

Type: core

Students enrolled

Search student

Figure 24: Course without assigned students

10.1.4 Duplicate Assignment Prevention

Before assigning:

- The system checks if the student is already enrolled in the course
- Prevents duplicate relationships

EDIT COURSE

[return](#)

Name

Credit Hour

MS Teams Link

Type

Students enrolled

Add student to course

Student has already enrolled this course

Figure 25: Duplicate assignment prevention

10.1.5 Invalid Operations Handling

The system prevents:

- Assigning non-existent students or courses
- Re-adding the same student/course
- Accessing empty arrays

ADD NEW STUDENT

[return](#)

Name

Danesh Lou

ID

BSW01084693

Email

daneshlou05@gmail.com

Phone number

0129027096

Student with the ID "BSW01084693" already exists

Figure 26: Invalid operation handling

11.0 Middleware API Implementation

11.1 Overview

A lightweight middleware-like mechanism is implemented using in-memory caching to simulate API behavior. This improves usability and reduces repeated data entry.

11.1.1 Data Caching

The system stores previously entered:

- Student names
- Course codes

This allows reuse of frequently used data without retyping.

VIEW ALL COURSES

[return](#)

[1] software construction

Code: csda1231312

Credit Hour(s): 1

MS Teams Link: <https://teams.microsoft.com/l/meetup-join/>

Type: core

Students enrolled

[1] Danesh Lou | BSW01084693 | daneshlou05@gmail.com | 0129027096

Search student

VIEW ALL STUDENTS

[return](#)

[1] Danesh Lou

ID: BSW01084693

Email: daneshlou05@gmail.com

Phone Number: 0129027096

Courses taken

[1] software construction | csda1231312 | 1 | <https://teams.microsoft.com/l/meetup-join/> | core

Search course

Figure 27: Data caching

Benefits

- Faster data entry
- Reduced user errors
- Improved interaction experience

12.0 System Integration

12.1 Overview

All components are integrated into a single system using a menu-driven interface, ensuring smooth interaction between modules.

12.1.1 Main Menu Interface

A loop-based menu allows users to:

1. Add new course
2. View all course
3. Search course
4. Edit course
5. Delete course
6. Add new student
7. View all student
8. Search student
9. Edit student
10. Delete student

This ensures all features are accessible in one system.

Course

- [Add new course](#)
- [View all courses](#)
- [Search course](#)
- [Edit course](#)
- [Delete course](#)

Students

- [Add new student](#)
- [View all students](#)
- [Search student](#)
- [Edit student](#)
- [Delete student](#)

Figure 28: Main Menu Interface

12.1.2 Integration Flow

1. User inputs data
2. System validates input (Error Handling – Part 3)
3. Data is processed and stored
4. Cache updated (Middleware – Part 4)
5. Output displayed

12.1.3 System Features

- Fully connected modules
- Data consistency across operations
- Clear input/output flow